Tabular Reinforcement Learning for "The Floor Is Slime!"

Alejandro León Martín Lama
Artificial Intelligence 2025/26
Universidad de Sevilla
Seville, Spain
alemarlam@alum.us.es

***Abstract*—This report presents a reinforcement learning solution for the long-jumps variant of the two-player race game "The floor is slime!". The problem was modeled as a finite stochastic decision process in which an agent must decide whether to stay, jump one tile, or jump two tiles while accounting for changing terrain, stochastic landing outcomes, and an opponent that also tries to reach the goal. The learning algorithm is tabular Q-learning, implemented from scratch without reinforcement learning libraries. The project includes a rule-based game engine, four dummy opponent strategies, a training and evaluation pipeline, policy persistence, and reproducible CSV/JSON experiment outputs. Particular attention was given to design choices that affected learning quality: the state representation, reward shaping, turn and round ownership, the use of a maximum round limit, and the choice of training opponents. Experiments show that a compact local state representation learns substantially better on the default 10-tile game than a full slime-mask state, reaching a mean win rate of 0.578 after 20,000 episodes compared with a 0.411 random baseline. The full state is more theoretically Markovian, but it suffers from severe state-space growth. On a 20-tile track, the full-state table visited more than five million states. The main conclusion is that tabular Q-learning is suitable for the assignment-sized game when supported by a compact state abstraction and carefully balanced rewards, but its scalability is limited.**

***Index Terms*—reinforcement learning, Q-learning, Markov decision process, tabular learning, stochastic games, reward shaping**

I. Introduction

Reinforcement learning (RL) studies how an agent can learn to act by interacting with an environment and receiving rewards or penalties [1]. Unlike supervised learning, the agent is not given the exact correct answer after an error. Instead, it receives an evaluation of its behavior and must discover a policy that maximizes long-term return. This is appropriate for games because an action that looks good immediately may be bad later, and an action that is risky may still be necessary to win.

The selected assignment game is "The floor is slime!". It is a two-player race on a line of tiles. Each player starts at the first tile, and the first player to reach the final tile wins. Interior tiles may be dry or slimy, and after every round empty tiles may flip between both terrain types. The selected variant is the long-jumps version. In this version a player may stay, jump one tile, or attempt a two-tile long jump. The long jump is strategically important because it can win races quickly, but it has a lower landing probability, especially when the target tile is slimy.

The objective of this project is to implement a complete RL pipeline for this game from first principles, following the requirements of the course assignment [2]. The assignment requires an original algorithmic implementation, a simulated dummy player for training, a way for a user to play against computer-controlled behavior, reproducible experiments, and a scientific report. The implementation therefore focuses on clarity and explainability rather than on neural-network scale. A tabular Q-learning method was chosen because the action space is very small, the values can be inspected directly, and the update equation can be defended clearly.

Several design decisions were refined during development. First, game logic was moved into `game.py` so that turn order and round advancement belong to the environment, not to the learning loop. Second, state representation was studied in two forms: a full slime-mask representation and a compact reduced representation using local landing information. Third, reward shaping was adjusted after observing that excessive fall penalties encourage behavior that is too conservative for a race. Fourth, a maximum round limit was introduced so that long or stalled games are counted as timeouts instead of running indefinitely.

The report follows the structure requested by the assignment. Section II gives the RL background. Section III describes the game and the MDP formulation. Section IV presents the implementation architecture. Section V explains the Q-learning algorithm. Section VI describes opponents and training. Section VII discusses reward design. Section VIII presents the experimental methodology. Section IX analyzes the CSV results. Section X discusses the main lessons. Sections XI–XIII describe limitations, reproducibility, and future work, and Section XIV concludes.

II. Reinforcement Learning Background

An RL problem is commonly formalized using the assumptions of a Markov Decision Process (MDP): the agent perceives a finite set of states, has a finite set of available actions, acts in discrete time, and receives a numerical reward or penalty from the environment [1]. At each decision step, the agent observes a state s, selects an action a, receives a

reward r, and transitions to a new state s'. The goal is to learn a policy π that maximizes expected discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \tag{1}$$

where $\gamma \in [0, 1]$ is the discount factor.

The Markov property means that the current state contains all information needed to model the distribution of future states given the chosen action. In practice, implementations often use a state abstraction. A compact abstraction may violate the exact Markov property, but it can generalize better when tabular data is sparse. This tradeoff became central in this project.

Q-learning is a reinforcement learning method in which the agent assigns values to state-action pairs [1]. The value $Q(s, a)$ estimates the long-term usefulness of taking action a in state s, combining the immediate reward with discounted future rewards. The update rule used in this project is the gradual version of this idea:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s,a) \right]. \tag{2}$$

For terminal states, the future value term is omitted. The learning rate α controls how strongly the new sample changes the old estimate, while γ controls how much future rewards influence the current value. The main alternative considered was SARSA, an on-policy method that updates using the action actually selected next. SARSA could be useful in this game because it may learn more conservative policies under exploration. However, Q-learning was kept as the main algorithm because it is simpler, standard, and easier to explain in a first implementation.

III. Game and MDP Formulation

A. Game Rules

The default game uses a track of length 10. Tiles are indexed from 0 to 9. Both players start on tile 0, and tile 9 is the goal. Player 0 is the learner and player 1 is controlled by a dummy opponent during training and evaluation. The game ends immediately when either player reaches the goal.

Every non-terminal, non-start tile may be dry or slimy. At initialization, each interior tile is independently sampled as slimy with probability 0.5. After both players have acted in a round, each empty interior tile has probability 0.5 of flipping to the opposite terrain. Occupied tiles do not flip. This rule matters because waiting may preserve a currently favorable or unfavorable board, while moving may expose the agent to future terrain changes.

The available actions are:

- `STAY`: remain on the current tile.
- `JUMP`: attempt to move one tile forward.
- `LONG_JUMP`: attempt to move two tiles forward.

The landing probabilities are shown in Table I. If a jump fails, the player returns to tile 0. The goal and start tiles are represented as non-slimy in the board model.

TABLE I
Landing probabilities

Target terrain	`JUMP`	`LONG_JUMP`
Dry	0.90	0.70
Slimy	0.60	0.35

B. Action Space

The action set is:

$$A = \{\texttt{STAY}, \texttt{JUMP}, \texttt{LONG\_JUMP}\}. \tag{3}$$

The legal action set depends on the current player position. A one-tile jump is legal unless the player is already at the goal. A long jump is legal only when the player is at least two tiles away from the goal. Terminal states have no legal actions. The implementation enforces legal actions in the game engine rather than in the learning code, so all players share the same rule validation.

C. Transition Dynamics

The transition is stochastic for two reasons. First, jump outcomes are sampled by a Bernoulli trial determined by the target terrain and action type. Second, after both players act, slime flips are sampled independently for empty interior tiles. A transition from one learner decision state to the next therefore includes the learner's landing outcome, the opponent's chosen action and landing outcome, and the terrain flips at the end of the round.

This structure motivated an important design decision: the learning update is made after a complete round, not immediately after the learner action. The agent's reward and next state are computed after the opponent has responded and the terrain has changed. This better matches the strategic unit of the game. A move is good not only because it advances the learner, but also because it leaves the learner in a favorable state after the opponent has had a chance to win or change the race tempo.

D. State Representation

Two state encodings were considered.

The first encoding was the full state:

$$s_{\text{full}} = (p_0, p_1, m), \tag{4}$$

where p_0 is the learner position, p_1 is the opponent position, and m is a binary mask for all slime tiles. This representation is closest to the formal MDP because it includes the entire terrain configuration. For a track of length L, the approximate upper bound is:

$$|S_{\text{full}}| \approx L^2 2^{L-2}. \tag{5}$$

The second encoding was the reduced state:

$$s_{\text{red}} = (p_0, p_1, t_0^1, t_0^2, t_1^1, t_1^2), \tag{6}$$

where t_i^1 and t_i^2 indicate whether player i's one-tile and two-tile landing targets are slimy. This state is not a full Markov state because it does not include terrain farther away. However, it captures the information that most directly affects the next action and dramatically reduces the number of Q-table entries. This was not an accidental shortcut: it was an explicit state abstraction chosen to test whether generalization through abstraction could outperform exact but sparse tabular learning.

The experiments show that this tradeoff matters. On the 10-tile track, the reduced state reached 771 known states after 20,000 episodes and performed best. The full state reached 17,983 known states but performed worse. On a 20-tile track, the full state reached 5,203,483 known states, showing the curse of dimensionality very clearly.

IV. Implementation Architecture

The source code is intentionally small and organized by responsibility. The main files are listed in Table II. The implementation uses only the Python standard library for the learning algorithm. No external RL-specific libraries are used.

TABLE II
Main source files

File	Responsibility
`game.py`	Core rules, actions, turn order, rounds
`dummy_players.py`	Fixed opponent strategies
`rl.py`	Q-learning, rewards, train/evaluate, JSON policies
`experiments.py`	Reproducible experiment runner
`dummy_play.py`	Human or dummy terminal game interface
`play_agent.py`	Human against saved JSON policy

A. Game Engine

The game engine defines the `Action` enumeration. Each action stores its jump distance and landing probabilities. Keeping these constants inside the action definition makes the rule model easy to inspect and avoids duplicating probability values in the learning code.

The `Game` object stores the mutable environment state: player positions, slime positions, the winner, the round number, and the current player. During development, some round logic was temporarily repeated in `rl.py`. This was later refactored so that `apply_action()` advances the game automatically. When player 0 acts, the current player changes to player 1. When player 1 acts, empty tiles flip, the round counter increases, and the current player resets to player 0. This design is better because there is only one source of truth for the rules. The RL code no longer needs to remember when tile flipping should occur.

One consequence of this design is that the game engine also enforces turn order. Calling `legal_actions()` for the wrong player returns an empty tuple, and calling `apply_action()` out of turn raises an error. This prevents invalid training trajectories from silently entering the Q-table.

B. Policy and Q-Table Storage

The Q-table is a Python dictionary from state keys to dictionaries of action names and floating-point values. A typical entry stores values for `STAY`, `JUMP`, and `LONG_JUMP`. Policies are saved as JSON files containing the Q-table, the training configuration, and the reward configuration. JSON was chosen because it is human-readable, easy to generate with the standard library, and convenient for checking whether policies contain a reasonable number of known states.

A bug discovered during review was that using `q_table or {}` in the learner constructor replaced an intentionally shared empty dictionary with a new dictionary. This meant the learner appeared to run but did not preserve learning between episodes. The constructor was corrected to use the provided table whenever it is not `None`. This debugging step is important because it shows why policy size and smoke tests are useful: an empty Q-table can still play by random tie-breaking, but it is not a learned policy.

C. Human Play Against a Learned Policy

The project includes a dedicated command-line script, `play_agent.py`, for playing against a saved JSON policy. This script is separate from the training code because interactive play and experiment execution have different needs. The experiment runner must simulate many games without input, while the play script must display the board, ask for human actions, describe moves, and stop cleanly when the game ends or reaches the maximum round limit.

The script loads the JSON policy file, reads the saved Q-table and training metadata, and creates a `QLearningPlayer` attached to a new `Game` instance. If the user does not pass a track length or maximum round limit, the script reuses the values stored in the policy metadata. This avoids an important class of errors: a policy trained on a 10-tile track should not accidentally be used on a different track length unless the user explicitly requests it.

By default, the learned policy is assigned to player 1 and the human is assigned to player 2. This is deliberate. The Q-table was trained from the perspective of player index 0, so letting the policy move first keeps the interactive behavior closest to the training setup. The script still provides a `--human-player` option, but changing the move order should be interpreted carefully because the state keys are perspective-based and the policy was not trained symmetrically for both player indices.

During play, `play_agent.py` uses the same board display and input helpers as the terminal dummy-player game. Human controls are `s` or `0` for stay, `j` or `1` for a one-tile jump, and `l` or `2` for a long jump. The learned policy acts greedily with respect to its Q-table: it does not use exploration during human play. If the current state was not seen during training, all action values default to zero and the agent breaks ties randomly among legal actions. This fallback is useful because it keeps the game playable, but it also makes unseen states easy to recognize during qualitative testing.

D. Maximum Round Limit

All training and evaluation functions now accept a maximum round parameter. The default is 200 for the 10-tile experiments. If a game reaches the limit without a winner, it is counted as a timeout. During training, a timeout is treated as a terminal transition for bootstrapping purposes: the final Q-update does not include future value beyond the cap. This prevents infinite or very long games and makes experiments reproducible. The CSV output includes wins, losses, timeouts, win rate, loss rate, timeout rate, and the maximum round value used.

V. Learning Algorithm

A. Q-Learning

The learning algorithm is tabular Q-learning. At the beginning of each episode, a new game is generated using a deterministic seed based on the base seed and episode number. The learner controls player 0. The opponent is selected according to the training-opponent setting. In the main experiments, training uses a mixed opponent setting that samples from random, cautious, and aggressive strategies.

The learner uses epsilon-greedy exploration. With probability ϵ, it chooses a random legal action. Otherwise, it chooses a legal action with maximum Q-value in the current state. This follows the exploration/exploitation tradeoff described in the Q-learning article: early in training, the agent should try a wider range of actions because its value estimates are still unreliable; later, it should rely more on the best values it has learned [1]. Ties are broken randomly. Random tie-breaking matters because all unseen actions initially have value 0. Without random tie-breaking, the agent could develop a persistent bias toward whichever action appears first in the enumeration.

The training loop can be summarized as follows:

1) Reset the game and construct the learner and dummy opponent.
2) Encode the learner decision state.
3) Select a learner action using epsilon-greedy exploration.
4) Apply the learner action through the game engine.
5) If the learner has not won, let the opponent choose and apply its action.
6) If the opponent has not won, the game engine flips tiles and advances the round.
7) Compute the reward for the round.
8) If the game ended or timed out, update without bootstrapping.
9) Otherwise, encode the next state and update using (2).

B. Hyperparameters

The main hyperparameters are shown in Table III. These values were chosen to keep the algorithm simple and stable. The learning rate is moderate: large enough to respond to new samples, but not so large that one unlucky failed jump completely overwrites previous experience. The discount factor values future positioning and eventual wins, which is necessary because most actions do not end the game immediately.

TABLE III
Main Q-learning hyperparameters

Parameter	Value
Learning rate α	0.15
Discount factor γ	0.90
Initial epsilon	0.30
Final epsilon	0.02
Epsilon decay	0.9995
Default track length	10
Default max rounds	200
Evaluation games	1000 per opponent

C. Why Q-Learning Was Chosen

Q-learning was selected because it matches the assignment constraints and the scale of the game. The state and action spaces are discrete, the action set has only three elements, and the value table can be saved and inspected. This makes the method transparent for a defense presentation. It is also simple enough to implement from scratch correctly.

Other methods were considered. SARSA could make the policy more cautious because it learns the value of the actually followed exploratory policy. Expected SARSA could reduce update variance. Double Q-learning could reduce overestimation of risky long jumps. Value iteration would be possible if all transition probabilities, opponent policies, and tile flips were enumerated, but that would shift the project toward planning rather than learning from simulated experience. For this assignment, Q-learning provides the best balance between originality, clarity, and sufficient performance.

VI. Dummy Opponents

The assignment requires simulated opponents because a student cannot manually play thousands of training games. Four dummy strategies were implemented:

- **Random**: chooses uniformly among legal actions.
- **Cautious**: `STAY` if the next tile is slimy; otherwise `JUMP`.
- **Aggressive**: uses `LONG_JUMP` whenever legal, otherwise `JUMP`.
- **Balanced**: uses `LONG_JUMP` when the two-tile target is dry and `JUMP` otherwise.

The strategies were designed to create different learning pressures. Random is a weak baseline. Cautious is slow but avoids many obvious risks. Aggressive is fast but often falls. Balanced is a stronger hand-coded policy because it uses terrain information while still moving quickly.

The main training setting uses a mixed opponent distribution over random, cautious, and aggressive opponents. Balanced is primarily used as a challenging evaluation opponent or as a possible specialized training opponent. This design tries to avoid overfitting to a single fixed opponent behavior.

VII. Reward Design

Reward design was one of the most important parts of the project. The game is a race, so the agent must learn that risk can be useful. A policy that never falls may still lose because it moves too slowly. A policy that always long-jumps may also

lose because failed jumps reset progress. The reward must therefore encourage winning and progress without making falling seem worse than losing the whole game.

The shaped reward used in the stored main result summaries is:

- win: $+10.0$
- loss: -10.0
- per-round step cost: -0.02
- progress reward: $+0.20$ per tile advanced
- fall penalty: -1.0

This reward is a compromise. The terminal values keep winning and losing as the main objective. The progress term gives intermediate feedback so the learner does not need to wait until the end of the game to learn that forward movement is usually useful. The step cost discourages waiting forever. The fall penalty is large enough to make failed jumps undesirable, but not so large that the agent is afraid of all risky movement.

During development, harsher reward settings were also considered, including configurations with a very large win reward and a fall penalty close to half the loss penalty. Analysis showed that such settings can make the immediate expected reward of even a dry one-tile jump worse than staying. That is not aligned with the race objective. This is why the recommended reward interpretation is balanced shaping, not maximum punishment for falling.

VIII. Experimental Methodology

A. Reproducibility

Experiments are run through `src/experiments.py`. The runner accepts the training budget, training opponent, evaluation opponents, number of evaluation games, random seed, track length, maximum rounds, and Q-learning hyperparameters. For every training budget, it saves a policy JSON file and writes CSV/JSON result summaries under `outputs/`.

The main command shape is:

```
./src/experiments.py
--episodes 1000 5000 20000
--eval-games 1000
--seed 42
--max-rounds 200
```

The 20-tile experiments use a higher maximum round limit:

```
./src/experiments.py
--episodes 20000
--track-length 20
--max-rounds 5000
```

B. Compared Conditions

The experiments compare:

- reduced-state learning on a 10-tile track;
- full-state learning on a 10-tile track;
- reduced-state learning on a 20-tile track;
- full-state learning on a 20-tile track.

For the 10-tile setting, policies were trained for 1,000, 5,000, and 20,000 episodes. For the 20-tile scalability test, policies were trained for 20,000 episodes. Each trained policy was evaluated over 1,000 games against the random, cautious, and aggressive opponents. A random-player baseline was also evaluated against the same opponents and seeds.

C. Metrics

The primary metric is win rate. The CSV files also record loss rate, timeout rate, known Q-table states, and improvement over the random baseline in percentage points. The number of known states is not a direct performance metric, but it is very useful for interpreting sample complexity. If a policy knows millions of states but performs only slightly above baseline, then the representation is likely too large for the training budget.

IX. Results and Analysis

A. Default 10-Tile Track

Table IV shows the reduced-state results for the default 10-tile game. At 1,000 episodes the agent is not yet reliable. It beats the random baseline against the cautious opponent by 1.4 percentage points, but it performs worse against random and aggressive opponents. By 5,000 episodes the policy is better than baseline in every opponent setting. By 20,000 episodes it achieves a mean win rate of 0.578, compared with a mean random baseline of 0.411.

TABLE IV
Reduced-state results on track length 10

Episodes	Opponent	Win	Baseline	Imp. pp
1,000	Random	0.417	0.500	-8.3
1,000	Cautious	0.408	0.394	+1.4
1,000	Aggressive	0.290	0.339	-4.9
5,000	Random	0.558	0.500	+5.8
5,000	Cautious	0.572	0.394	+17.8
5,000	Aggressive	0.425	0.339	+8.6
20,000	Random	0.626	0.500	+12.6
20,000	Cautious	0.651	0.394	+25.7
20,000	Aggressive	0.456	0.339	+11.7

The improvement is strongest against the cautious opponent. This is expected: cautious behavior avoids many falls but gives up tempo. A learned policy that is willing to take calculated jumps can exploit that slowness. The aggressive opponent remains the hardest of the three in the 10-tile reduced-state setting because it can win quickly when long jumps succeed.

B. Full State on Track Length 10

Table V shows the full-state results on the same 10-tile track. The 1,000 episode policy is slightly above baseline on average, but performance deteriorates at 5,000 and 20,000 episodes. At 20,000 episodes the mean win rate is 0.270, far below the reduced-state mean of 0.578.

The full-state result is the clearest evidence that exact state information is not automatically better for tabular learning. The full state has better theoretical properties, but it separates experiences that the reduced state shares. Two boards with identical local landing risks but different far-away slime

TABLE V
FULL-STATE RESULTS ON TRACK LENGTH 10

Episodes	Opponent	Win	Baseline	Imp. pp
1,000	Random	0.521	0.500	+2.1
1,000	Cautious	0.436	0.394	+4.2
1,000	Aggressive	0.334	0.339	-0.5
5,000	Random	0.326	0.500	-17.4
5,000	Cautious	0.238	0.394	-15.6
5,000	Aggressive	0.140	0.339	-19.9
20,000	Random	0.309	0.500	-19.1
20,000	Cautious	0.346	0.394	-4.8
20,000	Aggressive	0.154	0.339	-18.5

patterns become different Q-table keys. With only 20,000 episodes, many full-state keys receive too few updates to estimate values well.

C. Scalability to Track Length 20

The 20-tile experiments test scalability. Table VI compares the reduced and full representations after 20,000 episodes with a maximum round limit of 5,000.

TABLE VI
TRACK LENGTH 20 RESULTS AFTER 20,000 EPISODES

State	Opponent	Win	Baseline	Imp. pp	States
Reduced	Random	0.286	0.497	-21.1	4,556
Reduced	Cautious	0.056	0.093	-3.7	4,556
Reduced	Aggressive	0.210	0.388	-17.8	4,556
Full	Random	0.560	0.497	+6.3	5,203,483
Full	Cautious	0.107	0.093	+1.4	5,203,483
Full	Aggressive	0.432	0.388	+4.4	5,203,483

The reduced representation performs poorly on the longer track. Local information is no longer enough: planning over a longer horizon requires more knowledge about future terrain and accumulated race position. The full representation performs slightly above baseline, but only after visiting more than five million states. This is not practical as a final tabular design. The 20-tile results therefore support two conclusions at the same time: compact local state can work well for the default game, and full tabular state does not scale cleanly.

D. Known State Counts

Table VII summarizes the state-count tradeoff. The reduced 10-tile policy saturates at about 771 known states, while the full 20-tile policy reaches more than five million. The performance improvement from those millions of states is modest, so the extra table size is not justified for the assignment-sized solution.

X. DISCUSSION

A. What the Agent Learned

The best default policy learns a useful race strategy. It does not merely avoid falls; it learns that forward progress is necessary. Its strongest performance is against cautious opponents, which confirms that it can exploit slow play. Its weaker performance against aggressive opponents shows that stochastic long-jump success can still create pressure. This is appropriate for the game: a simple tabular learner should not be expected to dominate every hand-coded strategy.

TABLE VII
KNOWN Q-TABLE STATES AND MEAN PERFORMANCE

Experiment	Known states	Mean win
10 tiles, reduced, 20k	771	0.578
10 tiles, full, 20k	17,983	0.270
20 tiles, reduced, 20k	4,556	0.184
20 tiles, full, 20k	5,203,483	0.366

B. Reduced State Versus Full State

The reduced-state result is not a failure of MDP thinking. It is an example of an engineering tradeoff. A full MDP state is desirable because it contains complete terrain information. However, a tabular algorithm does not generalize between different keys. In a small training budget, a compact abstraction can be superior because it reuses experience across many similar boards. The default 10-tile results show this very clearly.

For a final assignment implementation, the reduced state is the more appropriate default. The full state should be presented as an experiment demonstrating the curse of dimensionality. If the project were extended to longer tracks, a better solution would be function approximation or a more carefully engineered feature representation, not a larger raw table.

C. Reward Interpretation

The reward must be interpreted as part of the model design, not as an objective truth. The true objective is to win the game. The shaped reward is a tool that helps temporal-difference learning assign credit before the final outcome. The fall penalty is especially sensitive. If it is too small, the agent may jump recklessly. If it is too large, the agent may learn that staying is safer than moving, even though a race cannot be won by waiting. The selected balanced reward therefore intentionally values progress and terminal wins more than accident avoidance.

D. Round Ownership

Moving round logic into the `Game` class improved the correctness of the implementation. Earlier versions of the learning loop manually applied both player actions and then manually flipped tiles. That duplicated game rules outside the environment. The final design makes `apply_action()` responsible for advancing turns and rounds. This is cleaner because dummy players, human play, and RL training all share the same transition mechanics.

XI. THREATS TO VALIDITY AND LIMITATIONS

A. Single Seed

The CSV files analyzed in this report use seed 42. This makes the experiments reproducible, but it is not enough to fully characterize stochastic learning. A stronger final study

should repeat each condition with several seeds, for example 7, 42, and 123, and report mean and standard deviation.

B. Policy File Naming

Some experiment CSV files point to the same policy file names even when the state representation or track length differs. This means later runs may overwrite earlier policies. The CSV rows themselves remain useful as recorded results, but final experiments should include the state mode and track length in policy file names.

C. Interaction Interface

The terminal interface supports human play against human, dummy, and learned policy opponents. The learned-policy path is intentionally simple: it loads a JSON policy, creates the same game environment used during training, and lets the human enter actions from the terminal. This satisfies the interaction requirement of the assignment and also provides a qualitative debugging tool. One limitation remains: because the policy was trained as player 0, the most faithful interactive setup is for the learned policy to move first. A future version could train mirrored policies or encode player perspective more explicitly so that either player order is equally natural.

D. Limited Opponent Models

The dummy opponents are fixed heuristics. They are useful for controlled evaluation, but they do not adapt. A learned policy trained only against such opponents may exploit their patterns without becoming generally strong. Mixed training reduces this risk but does not remove it.

E. No Function Approximation

The implementation is intentionally tabular. This makes the project transparent, but it limits scalability. The 20-tile full-state experiment demonstrates this clearly: millions of Q-table states are required, and performance improves only slightly over baseline.

XII. Reproducibility

The project can be reproduced using the scripts in `src/`. The most important command is:

```
./src/experiments.py
--episodes 1000 5000 20000
--eval-games 1000
--seed 42
--max-rounds 200
```

Outputs are written to:

- `outputs/policies/`: JSON Q-table policies.
- `outputs/results/`: CSV and JSON summaries.

The CSV files used for this report are:

TABLE VIII
CSV result files used in the report

Condition	File in `outputs/results/`
10 tiles, reduced	`reduced_state.csv`
10 tiles, full	`full_state.csv`
20 tiles, reduced	`track20_reduced.csv`
20 tiles, full	`track20_full.csv`

To play against a saved policy, the command is:

```
./src/play_agent.py
--policy
outputs/policies/
q_mixed_20000_seed42.json
```

The optional `--human-player`, `--seed`, `--track-length`, and `--max-rounds` arguments can be used to change the interactive setup. In the default configuration, the policy is player 1 and the human is player 2.

XIII. Future Work

Several extensions would improve the project. The first is multi-seed evaluation. The second is adding SARSA or Double Q-learning as comparison algorithms. SARSA would test whether on-policy learning produces safer behavior; Double Q-learning would test whether reducing overestimation improves long-jump decisions. The third is adding state-mode support as a first-class command-line parameter so reduced and full-state experiments do not require manual code edits. The fourth is adding more detailed behavioral statistics, such as fall rate, average rounds, action distribution, long-jump rate on dry targets, and long-jump rate on slimy targets. Finally, the interactive agent command could be extended with optional explanations of the Q-values behind each chosen action, which would make it more useful during the oral defense.

XIV. Conclusions

This project implemented a tabular Q-learning agent for the long-jumps variant of "The floor is slime!". The implementation models stochastic landing, changing terrain, turn order, dummy opponents, training, evaluation, policy saving, and timeout-aware experiments. The main design lesson is that exact state information is not always the best practical choice for a tabular method. On the default 10-tile game, the reduced state representation learned a stronger policy than the full slime-mask representation because it generalized across locally similar states. After 20,000 episodes, the reduced-state policy achieved a mean win rate of 0.578 across the evaluated opponents, outperforming the mean random baseline of 0.411.

The experiments also show that scalability is the central limitation. Full-state learning on a 20-tile track visited more than five million states. This is technically possible but not elegant or efficient. For larger games, the natural next step would be function approximation or a more structured feature representation, which is also the standard direction suggested when state spaces become too large for explicit Q-tables [1]. Within the assignment scope, however, tabular Q-learning with a compact state abstraction is an appropriate, explainable, and reproducible solution.

Use of Generative AI

Generative AI assistance was used as a support tool, not as the primary author of the project implementation. The game

logic in `game.py`, including the action model, player positions, tile flipping, turn progression, and round handling, was implemented manually by the student. Most of the Q-learning implementation in `rl.py`, including the Q-table structure, action selection, update rule, reward calculation, policy saving, and policy loading, was also manually implemented by the student.

The tool used was ChatGPT/Codex, a conversational programming assistant accessed inside the local development workspace. It was able to read project files, inspect terminal output, propose code changes, run small verification commands, and help rewrite documentation. The interaction was iterative: the student asked questions about the design, requested code reviews, made manual changes, and then used the assistant to point out inconsistencies or missing functionality.

The AI assistance was mainly used to review the implementation, identify possible bugs or design risks, and help build the experimental workflow around the core algorithm. In particular, it helped with experiment scripts, CSV/JSON result organization, timeout handling, interpretation of reduced-state versus full-state experiments, and drafting or restructuring parts of this report. It also helped add the command-line script for playing against a saved JSON policy. The tool was not used as a substitute for understanding the algorithm: the student kept control over the game rules, the main Q-learning design, the final design decisions, and the interpretation accepted for the report. All final code behavior and experimental conclusions were reviewed by the student and are understood before being accepted as the final project submission.

REFERENCES

[1] F. Sancho Caparrini, "Aprendizaje por refuerzo: algoritmo Q Learning," *Blog de Fernando Sancho Caparrini*, Universidad de Sevilla. [Online]. Available: https://www.cs.us.es/~fsancho/Blog/posts/Aprendizaje_por_Refuerzo_Q_Learning

[2] A. Riscos Nunez and J. M. Moyano Murillo, "Artificial Intelligence 2025/26: Main practical assignment: Reinforcement learning," Universidad de Sevilla, 2025.

[3] Python Software Foundation, "Python 3 standard library documentation." [Online]. Available: https://docs.python.org/3/library/